Pandas

Pandas è uno strumento di analisi e manipolazione dei dati veloce e flessibile basato sul linguaggio Python.
Pandas fornisce due tipi (classi) di strutture per la gestione dei dati: *Series* e *DataFrame*.

Series

Una Series è una array unidimensionale di dati che possono essere di qualsiasi tipo: intero, stringa, ecc. I dati sono presentati in colonna ed etichettati (di default, con numeri interi crescenti). La sintassi per creare un oggetto di tale classe è

```
pd.Series(data=None, index=None, dtype=None, name=None, copy=False, ...)
```

Per esempio:

In [11]:
```
import pandas as pd
s = pd.Series([1, 3, 14, 12, 6, 17])
s
```

Out[11]:
```
0     1
1     3
2    14
3    12
4     6
5    17
dtype: int64
```

In [12]:
```
M = [1, 2, 3, 4, 5, 6, 7]
pd.Series(M, index=("a","b","c","d","e","f","g"), dtype=float, name="corrispondenza
```

Out[12]:
```
a    1.0
b    2.0
c    3.0
d    4.0
e    5.0
f    6.0
g    7.0
Name: corrispondenza, dtype: float64
```

Se gli oggetti non sono omogenei, il tipo di dato assegnato è quello più generale *object*:

In [13]:
```
S = pd.Series([1, 3, "a", 12, 6, 17])
S
```

Out[13]:
```
0     1
1     3
2     a
3    12
4     6
5    17
dtype: object
```

La proprietà `values` della classe `Series` restituisce quest'ultima nella forma di una ndarray (quindi, tra l'altro, non etichetta):

In [14]:
```
S.values
```

Out[14]:
```
array([1, 3, 'a', 12, 6, 17], dtype=object)
```

DataFrame

Un DataFrame è una struttura di dati bidimensionale etichettata che contiene dati come una array bidimensionale o una tabella. Si tratta di una *serie multipla*. La sintassi per creare un oggetto di tale classe è

```
pd.DataFrame(data=None, index=None, columns=None, dtype=None, copy=None)
```

Il parametro `index` può essere usato per fornire indici per le righe. Se non si fornisce alcun argomento di input per tale parametro e non vi sono elementi riconosciuti come indici nei dati di ingresso, venogono usati i numeri 0, 1, 2, ... Analogamente, nomi per le colonne possono essere assegnati tramite `columns`.
`copy` può essere `None` (valore di default) o un `bool`. Come mostrato in uno degli esempi sottostanti, tale parametro consente di copiare dati da un input.

Come primo esempio, creiamo una tabella (più correttamente, una struttura tabulare del tipo DataFrame di Pandas) usando la funzione `random.rand` di NumPy vista precedentemente:

In [15]:
```
D =pd.DataFrame(np.random.rand(7,3))
D
```

Out[15]:

	0	1	2
0	0.462014	0.075251	0.569069
1	0.161540	0.756085	0.622166
2	0.017548	0.308295	0.253573
3	0.698896	0.469198	0.935843
4	0.161468	0.935995	0.291293
5	0.648442	0.073582	0.635155
6	0.793677	0.903793	0.348851

Assegniamo nomi a righe e colonne:

In [16]:
```
DD =pd.DataFrame(np.random.rand(7,3),
                 index=["$t_0$","$t_1$","$t_2$","$t_3$","$t_4$","$t_5$","$t_6$"],
                 columns=["$f_0$","$f_1$","$f_2$"])
DD
```

Out[16]:

	f_0	f_1	f_2
t_0	0.791289	0.953579	0.381904
t_1	0.522862	0.845412	0.760290
t_2	0.784316	0.762924	0.295014
t_3	0.924113	0.428574	0.626760
t_4	0.436753	0.345371	0.831685
t_5	0.675373	0.898139	0.642430
t_6	0.994104	0.532998	0.001014

Alcuni metodi e funzioni di Pandas

Il metodo head(n=5) restituisce le prime `n` righe (come impostazione predefinita, cinque righe) dell'oggetto di tipo `DataFrane` a cui si applica:

In [17]:
```
DD.head()
```

Out[17]:

	f_0	f_1	f_2
t_0	0.791289	0.953579	0.381904
t_1	0.522862	0.845412	0.760290
t_2	0.784316	0.762924	0.295014
t_3	0.924113	0.428574	0.626760
t_4	0.436753	0.345371	0.831685

In [18]:
```
DD.head(n=4)
```

Out[18]:

	f_0	f_1	f_2
t_0	0.791289	0.953579	0.381904
t_1	0.522862	0.845412	0.760290
t_2	0.784316	0.762924	0.295014
t_3	0.924113	0.428574	0.626760

Il metodo tail(n=5) funziona analogamente a `head` con riferimento alle ultime righe:

In [19]:
```
DD.tail(2)
```

Out[19]:

	f_0	f_1	f_2
t_5	0.675373	0.898139	0.642430
t_6	0.994104	0.532998	0.001014

Il metodo `info` stampa informazioni sul dato `DataFrame`:

In [20]:
```
D.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   0       7 non-null      float64
 1   1       7 non-null      float64
 2   2       7 non-null      float64
dtypes: float64(3)
memory usage: 300.0 bytes
```

Il metodo `describe` genera una statistica descrittiva:

In [21]:
```
DD.describe()
```

Out[21]:

	f_0	f_1	f_2
count	7.000000	7.000000	7.000000
mean	0.732687	0.681000	0.505585
std	0.202473	0.242714	0.294068
min	0.436753	0.345371	0.001014
25%	0.599117	0.480786	0.338459
50%	0.784316	0.762924	0.626760
75%	0.857701	0.871776	0.701360
max	0.994104	0.953579	0.831685

Si possono anche selezionare le righe di un dataframe che soddisfano una certa condizione:

In [22]:
```
L = DD[DD["$f_1$"] <= 0.6]
L
```

Out[22]:

	f_0	f_1	f_2
t_3	0.924113	0.428574	0.626760
t_4	0.436753	0.345371	0.831685
t_6	0.994104	0.532998	0.001014

Il metodo `to_excel` consente di esportare il dataframe in uno sheet di un file Excel di cui si assegna il nome:

In [23]:
```
DD.to_excel("DD.xlsx",'Sheet1')
```

Possiamo anche creare una tabella leggendola da un file. Per esempio, sotto facciamo tale operazione usando un file di tipo `tsv` (dall'inglese *tab-separated values*)

In [24]:
```
orders = pd.read_table('chipotle.tsv')
orders
```

Out[24]:

	order_id	quantity	item_name	choice_description	item_price
0	1	1	Chips and Fresh Tomato Salsa	NaN	$2.39
1	1	1	Izze	[Clementine]	$3.39
2	1	1	Nantucket Nectar	[Apple]	$3.39
3	1	1	Chips and Tomatillo-Green Chili Salsa	NaN	$2.39
4	2	2	Chicken Bowl	[Tomatillo-Red Chili Salsa (Hot), [Black Beans...	$16.98
...	...	...	...	...	...
4617	1833	1	Steak Burrito	[Fresh Tomato Salsa, [Rice, Black Beans, Sour ...	$11.75
4618	1833	1	Steak Burrito	[Fresh Tomato Salsa, [Rice, Sour Cream, Cheese...	$11.75
4619	1834	1	Chicken Salad Bowl	[Fresh Tomato Salsa, [Fajita Vegetables, Pinto...	$11.25
4620	1834	1	Chicken Salad Bowl	[Fresh Tomato Salsa, [Fajita Vegetables, Lettu...	$8.75
4621	1834	1	Chicken Salad Bowl	[Fresh Tomato Salsa, [Fajita Vegetables, Pinto...	$8.75

4622 rows × 5 columns

Possiamo stabilire il numero massimo di righe da mostrare tramite un'istruzione come la seguente:

In [25]:
```
pd.set_option('display.max_rows',3)
```

Adesso,

In [26]:
```
orders
```

Out[26]:

	order_id	quantity	item_name	choice_description	item_price
0	1	1	Chips and Fresh Tomato Salsa	NaN	$2.39
...	...	...	...	...	...
4621	1834	1	Chicken Salad Bowl	[Fresh Tomato Salsa, [Fajita Vegetables, Pinto...	$8.75

4622 rows × 5 columns

La funzione reset_option risetta una o più opzioni al loro valore di default. Per esempio, con riferimento al numero massimo di righe da mostrare, l'istruzione è

In [27]:
```
pd.reset_option('display.max_rows')
```